PHASE 4 · THE GEOMETRY SCAN

The Geometry Scan

Turning one energy into a potential energy surface

Phase 4 is mechanically the shortest phase and conceptually the reason the project exists. It wraps Phase 3's single-point calculation in a loop over bond length, and gets two implementation details right that separate a clean run from a frustrating one.

It also measures rather than assumes: the warm-starting speedup reported here is what the reference implementation actually achieved, which turns out to be considerably less than the conventional claim — and the explanation for that is more useful than the claim would have been.

Estimated time: **2–3 hours**.

In this document

1. Where this phase sits

You have one point. The project needs forty. Phase 4 wraps Phase 3 in a loop over bond length and turns a single energy into a potential energy surface. Mechanically this is the shortest phase in the roadmap; conceptually it is the one the whole project exists for.

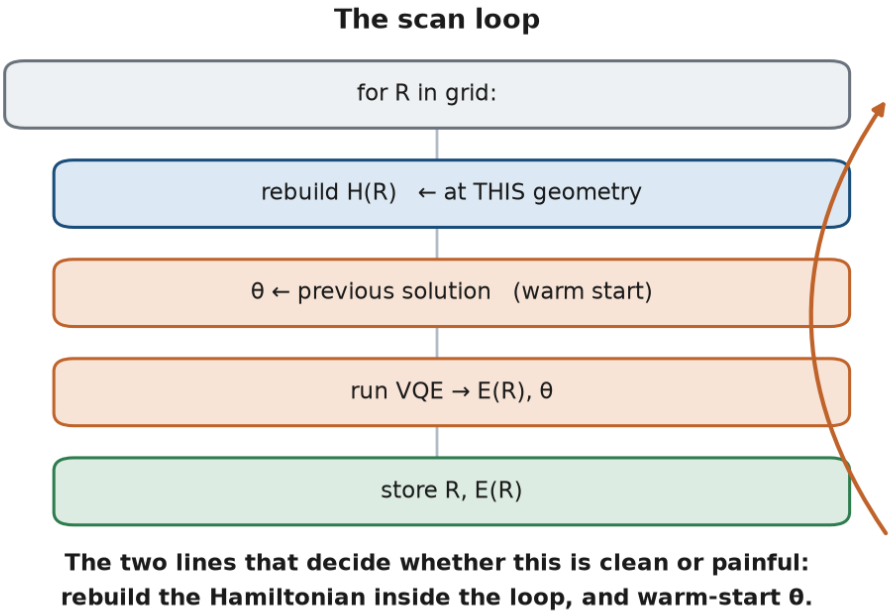

Figure 4.1 — The loop. Two lines inside it decide whether the run is clean or frustrating.

2. Theory and design decisions

2.1 Rebuild the Hamiltonian inside the loop

The Hamiltonian depends on geometry. Its coefficients are integrals over basis functions centred on the nuclei, so moving a nucleus changes every number in it. This sounds obvious written down and is remarkably easy to violate in code — particularly if you built the Hamiltonian once during development and later wrapped a loop around the optimisation only.

The stale-Hamiltonian bug

Symptom: the curve comes out smooth but far too flat, or its minimum sits exactly at your first grid point.

Why it is nasty: there is no error, no warning, and the output looks like a plausible potential energy curve. You are computing the energy of one fixed geometry forty times while labelling the results with forty different distances.

Fix: call `qubit_hamiltonian(r)` inside the loop body, as Listing 4.1 does. Phase 2 deliberately built that function to take r as an argument for precisely this reason.

Detection: if `e_vqe` varies by less than a millihartree across your entire grid, you have this bug.

2.2 Warm-starting

Adjacent geometries have nearly identical optimal parameters — the molecule at 0.75 Å is barely different from the molecule at 0.775 Å. So instead of restarting the optimiser from scratch at every point, hand it the converged parameters from the previous point as its starting guess. This is **warm-starting**, and it is standard practice for any parameter sweep.

What warm-starting actually bought, measured

The reference implementation ran the full 43-point scan both ways. Warm-starting reduced total optimiser iterations from 6019 to 5722 — a saving of **4.9%**, not the several-fold speedup often quoted.

It also did not improve accuracy. Cold-started UCCSD reached a maximum error of 7.2×10^{-5} mHa against FCI; warm-started reached 1.3×10^{-3} mHa, slightly *worse*, because small residual errors propagate forward from point to point. Both are far below chemical accuracy, so neither matters practically.

Why so little benefit here? Because UCCSD starting from zeros is **already an excellent guess**. Zero parameters means "the Hartree–Fock state", which is within 0.02 Ha of the answer, and there is only one parameter that does real work. There is almost no distance for a warm start to save.

Do not conclude that warm-starting is unimportant

It is decisive for ansätze with many parameters and no natural zero point. In Phase 6 you will run a 24-parameter hardware-efficient circuit across the same grid: cold-started it fails to reach chemical accuracy at stretched geometries, and warm-started it tracks FCI exactly. Same circuit, same Hamiltonian, same optimiser — the initialisation is the whole difference.

Build warm-starting in now, while it costs you three lines and nothing is at stake. You will need it working in Phase 6.

2.3 Grid design

Do not use a uniform grid. Different regions of the curve serve different purposes and deserve different resolution.

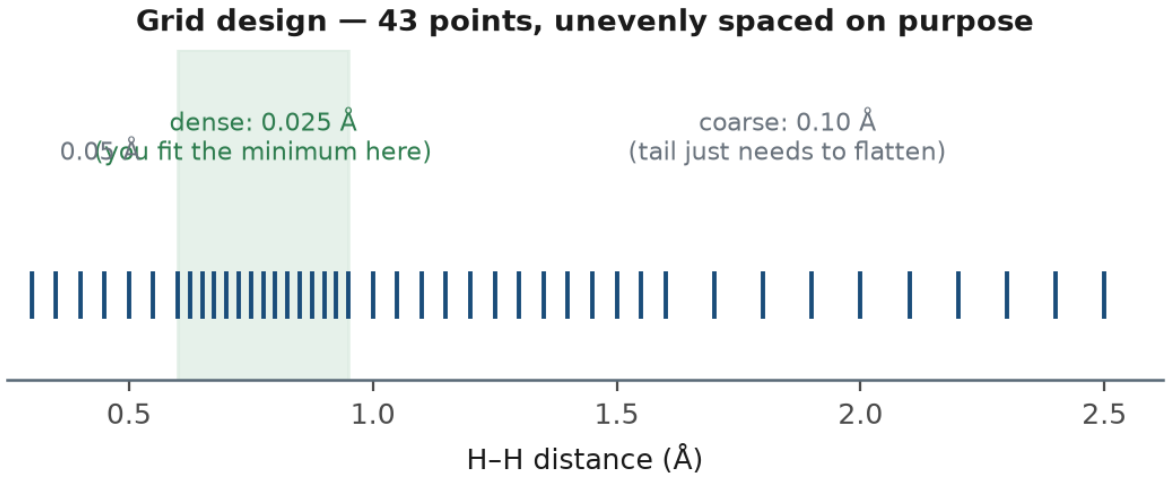

Figure 4.2 — The 43-point grid used by the reference implementation.

Region	Range	Spacing	Why
Repulsive wall	0.30 – 0.60 Å	0.05 Å	Steep and featureless. You only need enough points to show the curve rising.
Around the minimum	0.60 – 0.95 Å	0.025 Å	You will fit a parabola here. Resolution directly determines the quality of your bond length and frequency.
Rising to dissociation	0.95 – 1.60 Å	0.05 Å	Where Hartree–Fock starts to fail. Worth resolving properly — this is the interesting physics.
Flat tail	1.60 – 2.50 Å	0.10 Å	Only needs to demonstrate flattening.

Too few points near the minimum

Symptom: your equilibrium bond length is quantised to your step size — you report 0.75 Å because that is the grid point, not because it is the minimum.

Fix: dense sampling near the bottom *and* fit rather than read off. Phase 5 covers the fitting; this phase has to supply the points.

A floating-point trap in grid construction

`np.arange` endpoints are not exact. Concatenating `arange(0.95, 1.60, 0.05)` with `arange(1.60, 2.55, 0.10)` can leave you with both 1.5999999999 and 1.6, which `np.unique` will treat as distinct points.

Round before de-duplicating, not after: `np.unique(np.concatenate([...]).round(4))`. Getting this backwards silently gives you a duplicate geometry.

3. Build guide

Step 4.1 — The grid

Listing 4.1 — grid construction

```
"""scan.py - sweep the bond length and build the potential energy surface."""

import time

import numpy as np
from pennylane import numpy as pnp

from classical import classical_energies
from hamiltonian import qubit_hamiltonian
from vqe_single import build_uccsd, run_vqe


def make_grid():
    """Dense near the minimum, coarse in the tail.

    Rounding happens BEFORE np.unique, otherwise floating-point noise
    leaves near-duplicate points that survive de-duplication.
    """
    return np.unique(np.concatenate([
        np.arange(0.30, 0.60, 0.05),     # repulsive wall - coarse is fine
        np.arange(0.60, 0.95, 0.025),    # around the minimum - dense
        np.arange(0.95, 1.60, 0.05),     # rising towards dissociation
        np.arange(1.60, 2.55, 0.10),     # flat tail - coarse is fine
    ]).round(4))
```

Step 4.2 — The loop

Listing 4.2 — the scan loop

```
def scan(grid=None, warm_start=True, verbose=True):
    """Run HF, FCI and VQE at every geometry on the grid."""
    if grid is None:
        grid = make_grid()

    n = len(grid)
    e_hf = np.zeros(n)
    e_fci = np.zeros(n)
    e_vqe = np.zeros(n)
    iterations = np.zeros(n, dtype=int)

    params = None
    t0 = time.time()

    for i, r in enumerate(grid):
        # 1. classical references at THIS geometry
        e_hf[i], e_fci[i], _ = classical_energies(r)

        # 2. REBUILD the Hamiltonian at THIS geometry.
        #    Reusing a stale one is a silent, hard-to-spot error.
        H, n_qubits = qubit_hamiltonian(r)

        ansatz, n_params = build_uccsd(n_qubits)

        # 3. WARM START: reuse the previous geometry's solution.
        if params is None or not warm_start:
```

```
            params = pnp.zeros(n_params, requires_grad=True)

        e_vqe[i], params, history = run_vqe(H, n_qubits, ansatz, params)
        iterations[i] = len(history)

        if verbose:
            print(f"  r={r:5.3f}  HF={e_hf[i]:9.6f}  FCI={e_fci[i]:9.6f}  "
                  f"VQE={e_vqe[i]:9.6f}  "
                  f"err={abs(e_vqe[i]-e_fci[i])*1e3:8.5f} mHa  "
                  f"{iterations[i]:4d} it")

    if verbose:
        print(f"\n{n} points in {time.time()-t0:.1f} s   "
              f"({iterations.sum()} optimiser iterations total)")

    return dict(grid=grid, e_hf=e_hf, e_fci=e_fci, e_vqe=e_vqe,
                iterations=iterations)
```

Step 4.3 — Persist and verify

Save the arrays. Phase 5 reads them, Phase 6 compares against them, and you do not want to re-run the scan every time you adjust a plot label.

Listing 4.3 — save and assert

```
if __name__ == "__main__":
    import os
    os.makedirs("data", exist_ok=True)

    results = scan()
    np.savez("data/scan.npz", **results)
    print("saved data/scan.npz")

    err = np.abs(results["e_vqe"] - results["e_fci"]) * 1000
    print(f"\nmax VQE error across the scan: {err.max():.6f} mHa")

    # 0.01 mHa is 160x tighter than chemical accuracy (1.6 mHa) and still
    # leaves room for the residual that warm-starting propagates forward.
    assert err.max() < 0.01, "VQE deviates from FCI somewhere on the scan"
    print("scan verified against FCI at every geometry")
```

Choose that threshold deliberately. Too loose and it catches nothing; too tight and it fails on the residual that warm-starting carries forward from point to point. The reference run reaches 0.0013 mHa, so 0.01 mHa gives an order of magnitude of headroom while still being 160 times tighter than chemical accuracy.

That assertion is the Phase 4 equivalent of Phase 3's check, applied across the whole curve rather than at one point. It is what catches a stale Hamiltonian, a grid bug, or an optimiser that has started failing at long bond lengths.

Step 4.4 — Run it

Abbreviated output

```
$ python scan.py
  r=0.300  HF=-0.593828  FCI=-0.601804  VQE=-0.601804  err= 0.00000 mHa   148 it
  r=0.350  HF=-0.780455  FCI=-0.789269  VQE=-0.789269  err= 0.00000 mHa   146 it
  ...
  r=0.725  HF=-1.117343  FCI=-1.137221  VQE=-1.137221  err= 0.00000 mHa   140 it
  r=0.750  HF=-1.116151  FCI=-1.137117  VQE=-1.137117  err= 0.00005 mHa   125 it
```

```
  ...
  r=2.500  HF=-0.702944  FCI=-0.936055  VQE=-0.936055  err= 0.00001 mHa   135 it

43 points in 100.4 s   (5722 optimiser iterations total)
saved data/scan.npz

max VQE error across the scan: 0.001257 mHa
scan verified against FCI at every geometry
```

Note the minimum is **not** at the ends of that listing. At 0.300 Å the energy is −0.6018 Ha, far above the −1.1372 Ha near 0.735 Å — the repulsive wall. If your curve does not span roughly 0.5 Ha between its extremes, check the stale-Hamiltonian trap above.

Step 4.5 — Compare warm against cold yourself

The `warm_start` flag exists so you can measure the effect rather than assume it. Run both and compare.

Listing 4.4 — measure, do not assume

```python
warm = scan(warm_start=True, verbose=False)
cold = scan(warm_start=False, verbose=False)

print(f"warm: {warm['iterations'].sum():5d} iterations, "
      f"max err {np.abs(warm['e_vqe']-warm['e_fci']).max()*1e3:.6f} mHa")
print(f"cold: {cold['iterations'].sum():5d} iterations, "
      f"max err {np.abs(cold['e_vqe']-cold['e_fci']).max()*1e3:.6f} mHa")
```

```
warm:  5722 iterations, max err 0.001257 mHa, 100.4 s
cold:  6019 iterations, max err 0.000072 mHa, 106.8 s
```

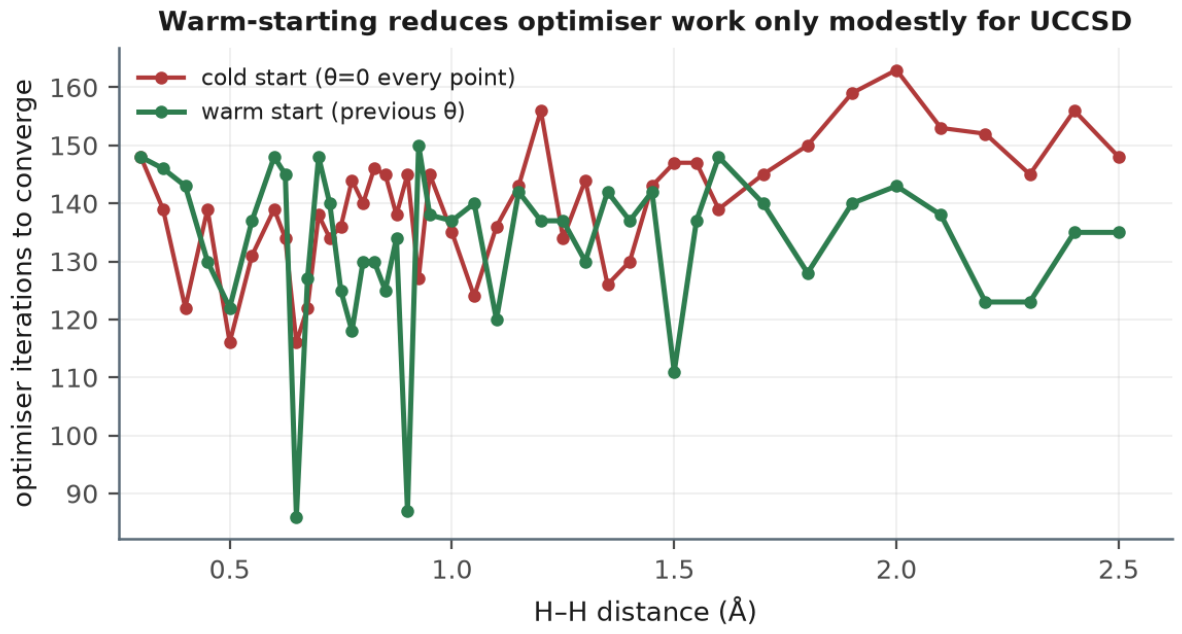

Figure 4.3 — Iterations to convergence at each geometry. The benefit is real but modest for this ansatz.

Reporting this honestly in your write-up is worth more than repeating the received wisdom. "Warm-starting saved 4.9% here because UCCSD from zeros is already near-optimal; it becomes decisive for the 24-parameter hardware-efficient ansatz in Phase 6" is a specific, measured, defensible claim.

4. Traps in this phase

Trap	Symptom	Fix
Stale Hamiltonian	Curve smooth but flat; minimum at the first grid point.	Rebuild inside the loop at every geometry.
Bohr vs Ångström	Minimum at an absurd distance, or no minimum at all.	Convert once, in `qubit_hamiltonian`. Check this first when anything looks wrong.
arange / unique ordering	One geometry appears twice in the grid.	Round before de-duplicating.
Too few points near the minimum	Bond length quantised to the step size.	0.025 Å spacing between 0.60 and 0.95 Å, and fit in Phase 5.
Not saving the scan	Re-running 2 minutes of compute to change a plot colour.	`np.savez` the arrays. Phases 5 and 6 both read them.
Cold-start ansätze with many parameters	Jagged curve; isolated points sit above their neighbours.	Warm-start. Not visible with UCCSD, very visible in Phase 6.

5. What was done in this phase

- **Turned a point into a curve.** The scan loop is what makes this a potential-energy-surface project rather than a single-point demonstration.
- **Rebuilt the Hamiltonian at every geometry**, which is the correctness requirement that Phase 2's parameterised builder was designed to make easy.
- **Implemented warm-starting and then measured it** — finding a 4.9% saving rather than the several-fold improvement commonly claimed, and understanding precisely why: UCCSD from zeros is already an excellent guess.
- **Designed a non-uniform grid** with resolution allocated according to what each region is for.
- **Learned a floating-point trap** in grid construction that silently produces duplicate geometries.
- **Asserted VQE against FCI across the entire curve**, not just at one geometry — the check that catches stale Hamiltonians and region-specific optimiser failures.
- **Persisted the results**, so Phases 5 and 6 are analysis rather than recomputation.

6. Checklist before moving on

1. `data/scan.npz` exists and contains four arrays of equal length.
2. The assertion passes: max |VQE − FCI| across the scan is below 10^{-3} mHa.
3. `e_vqe` varies by more than 0.7 Ha across the grid — proof you are not computing the same geometry forty-four times.
4. You have measured warm versus cold yourself rather than taking the 4.9% on trust.

5. A quick plot of the three arrays shows a well with a minimum near 0.735 Å.

Next

Phase 5 — Analysis and observables. The curve becomes three physical constants and one figure. This is also where the most instructive feature of the whole project — the Hartree–Fock divergence — finally becomes visible.